# Confluent Cloud Java Process Table Function (PTF) User-Defined Function (UDF) (early access release) — User Event Enricher

**Table of Contents**


- 1.0 State and operators in a Process Table Function
  - 1.1 What is state?
  - 1.2 How operators use state
  - 1.3 How `PARTITION BY` connects SQL to state
  - 1.4 The role of `@StateHint` and `@ArgumentHint`
- 2.0 What does this example do?
  - 2.1 Enrichment logic
  - 2.2 How it works end-to-end
  - 2.3 Key concepts illustrated
- 3.0 Project structure
- 4.0 How to run
  - 4.1 Prerequisites
  - 4.2 Deploy to Confluent Cloud (early access)
- 5.0 Resources


## 1.0 State and operators in a Process Table Function

A **Process Table Function (PTF)** is a new category of user-defined function introduced in Flink 2.1. Unlike scalar or aggregate UDFs, a PTF is a **stateful operator** — it sits inside the Flink dataflow graph just like a built-in operator (e.g., a windowed aggregation or a keyed process function), but you define its logic in a plain Java class.

### 1.1 What is state?

In stream processing, **state** is data that persists across events. Without state every event is processed in isolation — you can filter or transform it, but you cannot count things, detect sequences, or remember what happened before. State is what turns a stateless pipe into an intelligent processor.

Flink manages state for you: it stores it in a **state backend** (heap, RocksDB, etc.), checkpoints it for fault tolerance, and restores it on failure. You never serialize or recover state manually.

### 1.2 How operators use state

Every Flink operator that needs memory between events holds state. A few examples:

| Operator | State it keeps |
|---|---|
| Keyed window aggregation | Partial aggregates per key per window |
| Interval join | Buffered rows from both sides within the join window |

| Operator | State it keeps |
|---|---|
| Deduplication | Set of seen keys |
| **ProcessTableFunction (PTF)** | Whatever you declare via `@StateHint` — one instance per partition key |

The PTF is special because **you** decide what the state looks like. You declare a POJO, annotate it with `@StateHint`, and Flink handles the rest — creating one instance per partition key, persisting it in the state backend, and checkpointing it automatically.

### 1.3 How `PARTITION BY` connects SQL to state

When you write:

```
SELECT *
FROM TABLE(
    UserEventEnricher(
        input => TABLE user_events PARTITION BY user_id
    )
)
```

Flink does the following behind the scenes:

1. **Key the stream** — `PARTITION BY user_id` tells the Flink planner to hash-partition the input by `user_id`, exactly like a `keyBy()` in the DataStream API.
2. **Allocate isolated state** — for each distinct `user_id`, Flink creates a separate `UserState` POJO instance. For example, as shown in the sample data, Alice's state never overlaps with Bob's.
3. **Route events** — every incoming row is routed to the partition (and therefore the state instance) that matches its `user_id`.
4. **Call `eval()`** — your `eval()` method receives the row *and* the correct state instance already loaded. You read and mutate the POJO fields directly — no `ValueState.get()` / `.update()` boilerplate.
5. **Checkpoint** — Flink snapshots all state instances periodically. On failure it restores them and replays from the last checkpoint, so your PTF is exactly-once by default.

### 1.4 The role of `@StateHint` and `@ArgumentHint`

These two annotations are the bridge between your Java code and the Flink operator graph:

- `@StateHint` — marks a parameter as operator state. Flink sees the annotated POJO and wires it into the keyed state backend. Each `PARTITION BY` key gets its own instance, automatically serialized, checkpointed, and restored.
- `@ArgumentHint(ArgumentTrait.SET_SEMANTIC_TABLE)` — declares that the input argument is a *set-semantic table*, meaning the PTF acts as a keyed, stateful virtual processor over the entire partitioned stream. This is what distinguishes a PTF from a simple row-at-a-time scalar UDF.

Together, these annotations let you write what *looks* like a plain method but *executes* as a fully fault-tolerant, distributed, keyed-state operator inside the Flink pipeline.

---

## 2.0 What does this example do?

This example puts the above concepts into practice. The `UserEventEnricher` PTF reads raw user-interaction events from a Kafka topic (`user_events`), enriches each event with session and counting information, and writes the result to a second Kafka topic (`enriched_events`).

### 2.1 Enrichment logic

For every incoming event the function maintains three pieces of **per-user state** (one state instance per `PARTITION BY user_id` key):

| State field | Purpose |
|---|---|
| `sessionId` | Monotonically increasing session counter. Incremented each time a `"login"` event arrives. |
| `eventCount` | Running count of events **within the current session**. Reset to zero on each new session, then incremented for every event (including the login itself). |
| `lastEvent` | The `event_type` of the most recent event for that user. |

Each incoming row is emitted immediately with these three extra fields appended, so the output schema is:

```
user_id     STRING    — passed through automatically via PARTITION BY
event_type  STRING
payload     STRING
session_id  BIGINT    — which session this event belongs to
event_count BIGINT    — position of this event within the session
last_event  STRING    — the event type just processed
```

### 2.2 How it works end-to-end

```
Kafka (user_events)
        │
        ▼
   ┌──────────────┐
   │ user_events │   Flink SQL source table (JSON / latest-offset)
   └──────────────┘
        │
        ▼
  UserEventEnricher(input => TABLE user_events PARTITION BY user_id)
        │
        │  Per-user stateful enrichment:
        │    • login → new session, reset count
        │    • any event → increment count, record last_event
        │
        ▼
   ┌─────────────────┐
   │ enriched_events│   Flink SQL sink table → Kafka (enriched_events)
   └─────────────────┘
```

### 2.3 Key concepts illustrated

- `ProcessTableFunction` — the Flink 2.x API for stateful, set-semantic UDFs callable from SQL.
- `@StateHint` — declares a POJO whose fields Flink automatically persists per partition key, eliminating manual `ValueState` / `MapState` boilerplate.
- `@ArgumentHint(ArgumentTrait.SET_SEMANTIC_TABLE)` — tells Flink the input is a keyed, stateful virtual processor (set semantics), not a simple row-at-a-time scalar function.
- `PARTITION BY` — the SQL-side mechanism that keys the input table so each `user_id` gets its own isolated state instance.
- **Configurable Kafka bootstrap servers** — resolved in order: `KAFKA_BOOTSTRAP_SERVERS` env var, Flink config key `kafka.bootstrap.servers`, then `localhost:9092` default.

## 3.0 Project structure

```
examples/ptf_udf/cc_java/
├── app/
│   ├── build.gradle.kts                # Gradle build (Flink 2.1.0, Java
17)
│   └── src/main/java/ptf/
│       └── UserEventEnricher.java      # The ProcessTableFunction
implementation
├── gradle/wrapper/
│   └── gradle-wrapper.properties       # Gradle wrapper version config
├── gradlew                             # Gradle wrapper script (Unix)
├── gradlew.bat                         # Gradle wrapper script (Windows)
└── settings.gradle.kts                 # Gradle settings (project name)
```

## 4.0 How to run

All commands below are run from the **project root** (where the `Makefile` lives). Run `make help` at any time to see every available target.

### 4.1 Prerequisites

- macOS with Homebrew and Docker Desktop running
- Java 17+

Install all required tooling (including Gradle) if you haven't already:

```
make install-prereqs        # installs docker, kubectl, minikube, helm,
gradle, envsubst
```

### 4.2 Deploy to Confluent Cloud (early access)

To build and deploy the PTF UDF to Confluent Cloud, run the `deploy-cc-java-ptf-udf` Make target. You must supply a **Cloud API Key** (not a Cluster API Key). Create one with:

```
confluent api-key create --resource cloud
```

Then deploy with:

```
make deploy-cc-java-ptf-udf \
  CONFLUENT_API_KEY='<your-cloud-api-key>' \
  CONFLUENT_API_SECRET='<your-cloud-api-secret>'
```

> **Important:** Wrap the `CONFLUENT_API_KEY` and `CONFLUENT_API_SECRET` values in **single quotes** to prevent the shell from interpreting special characters (e.g., `+`, `/`) in the secret.

To tear down the deployment:

```
make teardown-cc-java-ptf-udf \
  CONFLUENT_API_KEY='<your-cloud-api-key>' \
  CONFLUENT_API_SECRET='<your-cloud-api-secret>'
```

## 5.0 Resources

- Create a User-Defined Function with Confluent Cloud for Apache Flink